RepoBuddy — Product Requirements Document

Oct 26, 2026 · @KHAJA

Summary

RepoBuddy is a desktop code explorer and editor that turns any repository into a map you can zoom through, question and edit. It is an entry for the Meta Global AI Developer Hackathon, indie developer track, due **Nov 4, 9:00 PM ET (Nov 5, 7:30 AM IST)**.

The problem. Understanding an unfamiliar codebase is slow. You open folder after folder, grep for names and piece the flow together in your head. Existing "graph + chat" tools draw a hairball of hundreds of file nodes and give text answers that are disconnected from the code.

The pitch. A map for your codebase. Zoom from folders to files to code, ask a question and watch the answer light up as a path through the code, and click anything on your running app to jump to the code behind it. It works on any repo you already have cloned, and keeps itself up to date as you edit, pull, switch branches and commit.

What makes it different:

- **Two zoomable views.** A Folder view you drill into level by level, and a Connections view showing how files depend on each other, both readable rather than a hairball.
- **Answers drawn as paths.** Every answer is a highlighted route through the graph, with each step linked to exact lines in the editor.
- **Point-to-code (add-on).** Click a button in a screenshot of your app and RepoBuddy finds the component that renders it, using SAM 3.1 plus Muse Spark.
- **Private mode.** An open-weights Meta model (Muse Glimmer) runs on a GPU you control, so private code never goes to a third-party AI service.
- **A real editor.** Monaco (the editor inside VS Code) lets you read, edit and save without switching apps.

Goals, non-goals and success metrics

The goal is a working demo where a newcomer understands a real repo in under five minutes, with every AI claim traceable to real lines of code.

Goals

1. Someone new to a repository can explain its main parts and one key flow within 5 minutes.
2. Every AI answer is grounded: each claim links to a file and line range, and anything unverified is labelled as such.
3. It works on real repos of up to about 2,000 files in TypeScript/JavaScript and Python.
4. Each Meta model does a distinct, visible job in the product.

Non-goals for the hackathon

- An autonomous coding agent that changes code on its own.
- A full VS Code replacement: no extensions, debugger, terminal or git UI.
- Every programming language.
- Private GitHub sign-in (OAuth). Public URLs and local folders only.
- Running any code from the repository.

Success metrics (targets to measure during the build; these numbers go in the demo and README)

Metric	How it is measured	Target
Indexing time	500-file repo, from open to map ready	under 2 minutes
Citation accuracy	20 questions you write about 3 repos; cited lines support the claim	18 of 20 or better
Path correctness	10 flows you trace by hand; highlighted path matches	8 of 10 or better
Point-to-code	10 UI elements across 2 apps; right component in top 3	7 of 10 or better
Answer latency	question to first highlighted path	under 20 seconds

Target users and use cases

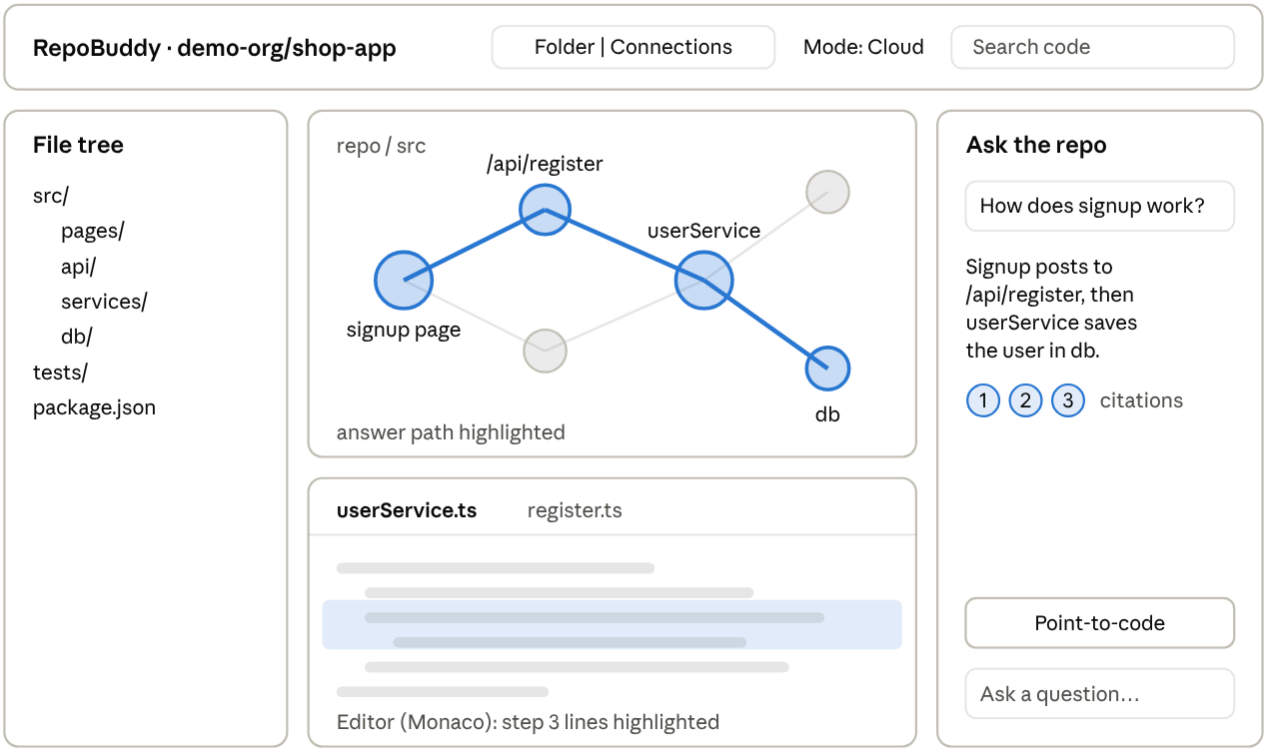
RepoBuddy is for individual developers and students who need to understand code they did not write, or no longer remember writing.

User	Situation	What they do in RepoBuddy
Student or new team member	Inherited a project or joined a hackathon team mid-way	Opens the Folder view, reads the folder summaries, asks "how does login work?"
Open-source newcomer	Wants to make a first contribution	Pastes the GitHub URL, asks "which files would I change to add X?"
Developer on their own large project	Forgot where things live	Searches a symbol, uses Connections view focus mode to see what depends on it
Frontend developer	Sees a button on screen and needs its code	Uses point-to-code on a screenshot
Developer at a company with strict rules	Cannot send code to cloud AI	Switches to Private mode

How it looks

RepoBuddy is one desktop window with four panes: file tree, graph, editor and chat, all linked so a click in one updates the others.

One window: map in the middle, code below, answers on the right



workspace layout · 4 panes

The graph and editor share the centre so the reader's eye moves straight from a highlighted step to its code; the accent colour is reserved for the answer path and its citations.

Screens

1. **Home:** a large input for a GitHub URL, an "Open folder" button, and a list of recently opened repos.
2. **Indexing:** progress through Scanning, Parsing, Linking, Summaries, with the file count. Opens the workspace as soon as parsing is done.
3. **Workspace:** the layout above. Panes are resizable; the graph and editor split can be swapped to graph-only or editor-only.
4. **Point-to-code panel:** opens over the chat panel. Drop zone for a screenshot, the click target, then the outlined element and the top 3 matching files.
5. **Settings:** Meta API key, mode (Cloud, Private, Hybrid), Spark tier, local model address (Ollama), folders to ignore.

Visual rules: dark theme by default; each top-level folder keeps the same colour in both views and the file tree; inferred links are dashed; unverified answer steps are shown in grey with a label.

Features and workflows

Eight features, built in priority order: F1 to F5 are the core product, F6 and F7 are the standout features, and F8 is a bonus.

F1. Open a repository

The user gets from a GitHub URL or folder to a ready map in one step.

1. On the home screen, the user pastes a public GitHub URL or picks a local folder.
2. RepoBuddy clones the repo (shallow clone, latest commit only) into a local cache.
3. It skips what never matters: anything in `.gitignore`, `node_modules`, build output, images, binaries and files over 1 MB.
4. A progress screen shows each stage: Scanning files, Parsing code, Linking imports, Writing summaries.
5. The workspace opens in the Folder view as soon as parsing finishes. Summaries keep filling in the background.

Stays in sync (live updates). RepoBuddy is not a one-time snapshot. For a local clone it watches the folder:

1. **You save a file** (in RepoBuddy or any other editor): that file is re-read and its arrows in the map update within a couple of seconds.
2. **You `git pull` or switch branches:** RepoBuddy notices the changed files, re-indexes them, and briefly highlights on the map what changed ("12 files changed in `api/` and `services/`"). This is a strong demo moment: pull a teammate's work, and see where it landed.
3. **You commit and push:** the files on disk are already up to date, so the map already shows your work; RepoBuddy records the commit so "what changed since my last visit" stays accurate.
4. Summaries are re-written only for files whose content actually changed (the hash cache), so updates stay fast and cheap.

F2. Folder view (structure map)

The Folder view shows the repo the way it is organised on disk, one level at a time, so it never becomes a hairball.

1. **Level 1:** each top-level folder is a node, plus the root files (README, package.json). Node size = lines of code inside.
2. **Click a folder** to zoom in. Its subfolders and files replace the view, and a breadcrumb appears at the top (`repo / src / api`).
3. **Click a breadcrumb** to zoom back out to any level.

4. **Double-click a file** to open it in the editor.
5. **Hover a node** to see its one-line AI summary ("Handles login, sessions and password reset").
6. **Arrows between folders** show how much they depend on each other, e.g. `api → services: 14 imports`. This means even the simple view shows how the parts talk.
7. **Badges** mark special files: entry point, config, tests, most-depended-on.

Stretch: a "Group by: Folders | Features" toggle where Muse Spark regroups files by what they do ("Auth", "Payments") instead of where they sit. Folders don't always match purpose; `utils/` is the classic example.

F3. Connections view (dependency graph)

The Connections view shows every file and how they depend on each other, with tools that keep it readable.

1. Every file is a node; every import is an arrow. Nodes are coloured by their top-level folder.
2. **Route links:** RepoBuddy matches frontend calls such as `fetch('/api/login')` to the backend route that handles `/api/login`. These are drawn as dashed arrows because they are inferred, not certain.
3. **Focus mode:** click a file to show only it and its neighbours one or two steps away. Everything else fades.
4. **Filters:** hide tests, config or a folder; filter by language.
5. **Simplified form:** a "Core only" toggle shows the 30 most-connected files, and a "Collapse folders" toggle merges files back into folder boxes (the Folder view's level of detail).
6. **Search:** type a file or function name to fly to it.

The two views are linked: selecting a node in one selects it in the other and in the file tree.

F4. Ask the repo (answers drawn as paths)

Asking a question returns a written answer and a highlighted path through the code, with every step clickable.

1. The user types a question in the right-hand panel, e.g. "What happens when a user registers?"
2. Muse Spark works through the codebase using tools RepoBuddy gives it: search the code, look up a function, read lines of a file, list a file's neighbours in the graph.
3. Spark returns a structured answer: a short explanation plus an ordered list of steps, each with a file, a line range and a one-sentence claim.

4. **Check step:** RepoBuddy confirms every cited file and line range exists and contains the function it names. Steps that fail are marked "unverified".
5. **On screen:** the answer appears in the panel with numbered citation chips. In the graph, the steps light up in order as a path (page → API route → service → database), and everything else dims.
6. **Click a step or chip** to open the editor at those lines, highlighted.
7. **Follow-ups:** right-click any node and choose "Ask about this file". The question is pre-filled with that context.

Example questions to support: "How does authentication work?", "Where is the database configured?", "Which files would I change to add a new payment method?", "Why does this component call this API?", "How do I run this project?"

F5. Editor

RepoBuddy is a usable editor, not just a viewer, built on Monaco rather than written from scratch.

1. File tree on the left; files open in tabs with syntax highlighting, find and replace, and save.
2. **Select code → "Explain"** gives a plain-language explanation in the chat panel.
3. **"Show in graph"** jumps from the open file to its node in either view.
4. **AI edit:** the user asks for a change. Spark proposes it as a diff (before and after, side by side). The user accepts or rejects. Accepting writes the file and re-indexes it. Nothing is ever applied without that click, and no code is ever run.

F6. Point-to-code (add-on, built after the core)

In plain terms: show RepoBuddy a screenshot of your running app, click the thing you are curious about, and it opens the code that draws it.

1. The user pastes or drops a screenshot of their app into the point-to-code panel.
2. They click an element, e.g. the "Add to cart" button.
3. RepoBuddy sends the screenshot to **SAM 3.1** with a fixed list of short UI prompts ("button", "text field", "card", "navigation bar", "image", "icon"). SAM returns an outline (mask) for every match.
4. RepoBuddy picks the outline that contains the clicked point and crops the screenshot to it. If none contains the click, it falls back to a box around the click.
5. **Muse Spark** reads the crop and returns what it sees: visible text ("Add to cart"), element type, and a short description.

6. RepoBuddy searches the index for that text and similar names (component names, labels, translation keys), and gathers candidate files.
7. Spark ranks the candidates and returns the top 3 with a reason each. The best match opens in the editor, and its node is highlighted in the graph.

Day-1 check: try SAM 3.1 on a few UI screenshots in Meta's playground to confirm these prompts work on interface elements.

F7. Private mode

In plain terms: normally RepoBuddy sends your code to Meta's cloud to be understood. In Private mode, a**n open-weights Meta model running on a machine you control (your own GPU, or your own private cloud GPU)** does that work instead, so your code never goes to a third-party AI service. That matters because many companies forbid pasting code into cloud AI tools.

A switch in the top bar has three settings:

Setting	What runs where	When to use it
Cloud	Muse Spark (API) does summaries and answers	Public repos; best quality
Private	Muse Glimmer on a GPU you control does everything	Code that must not go to a third-party AI service
Hybrid	Glimmer writes summaries privately; for a hard question, RepoBuddy shows you the exact snippets it wants to send to Spark and asks before sending	Best of both

Decision (Oct 27): the build laptop has 16 GB RAM and an RTX 3050 Laptop GPU, too small for a 30B model even at 4-bit (about 18–20 GB). Glimmer therefore runs on a private Modal endpoint using the hackathon's Modal credits. The demo says "an open-weights Meta model on a GPU you control; your code never goes to a third-party API", never "on this laptop". A user with a bigger GPU or Mac can point RepoBuddy at a local Ollama instead; only the endpoint address changes.

F8. Voice questions (bonus)

Hold a key and ask a question out loud. **Muse Voice Transcribe** turns speech into text as you talk. Its keyword biasing is fed the repo's own function and file names, so "useAuthStore" is transcribed correctly instead of as "use off store".

Meta AI models: role and significance

Five Meta models each do a different job. Muse Spark is the brain, SAM 3.1 is the eyes for screenshots, Muse Glimmer is the private local brain, and Voice Transcribe and Muse Wonder are optional extras.

Model	How it is accessed	Role in RepoBuddy	Why this model	Priority
Muse Spark 1.3	Meta Model API	Answers questions and traces flows using tool calls (F4); writes summaries in Cloud mode; reads screenshot crops and ranks candidates (F6); proposes edits (F5)	Built for agentic workflows with reliable tool calling; 1M-token context; native image and document input; Meta lists codebase understanding (SWEAtlas Codebase QnA) among its benchmarks	Must have
SAM 3.1	Meta Model API	Finds and outlines UI elements in a screenshot so the clicked element can be isolated (F6)	Detects objects from a short text prompt and returns pixel-precise masks in one call, with no training	Add-on
Muse Glimmer	Open weights, run locally (Ollama / LM Studio)	Private mode: file and folder summaries and Q&A without sending code anywhere (F7)	30B parameters, Apache 2.0, runs on a single GPU or a Mac; tuned for tool use	Should have
Muse Voice Transcribe	Meta Model API	Spoken questions (F8)	Streaming transcription with keyword biasing, so repo-specific names come out right	Bonus

Model	How it is accessed	Role in RepoBuddy	Why this model	Priority
Muse Wonder	Meta Model API	Optional "Deep investigate" button for hard bugs across many files	Plans long tasks, delegates to sub-agents and recovers from errors; priced higher ($4 / $20 per million tokens in/out), so only on demand	Bonus

Deliberately not used:

- **Muse Image.** Generated pictures of the architecture would compete with the real, grounded graph and could show things that are not in the code.
- **Muse Spark Open.** Far too large to run locally (816B parameters), and it adds nothing over the Spark API here.

Muse Code (Meta's coding agent) is worth using to build RepoBuddy itself. Mention it in the README; it counts as using Meta's tooling.

Cost and data note. Spark has a cheap "contributor" tier ($0.10 / $0.20 per million tokens) whose data is used to improve Meta's products, and a standard tier ($1.25 / $4.25) whose data is not. Use the contributor tier only for public demo repos, and the standard tier or Private mode for anyone's private code. The hackathon's $150 free credits carry the same condition (content may be used for product improvement), so during the build test and demo only on public open-source repos, and never on internship or employer code. Showing this choice in Settings is a small sign of craft judges will notice.

Fallback if Meta Model API billing stays blocked (status Oct 27: account restricted until a card verifies). The rules accept "Meta Model API and/or Meta's open-weight models", so the whole product can run on open weights. RepoBuddy calls models through one OpenAI-compatible client whose provider is set in `.env` (`LLM_BASE_URL`, `LLM_MODEL`, `LLM_API_KEY`), so switching is a config change:

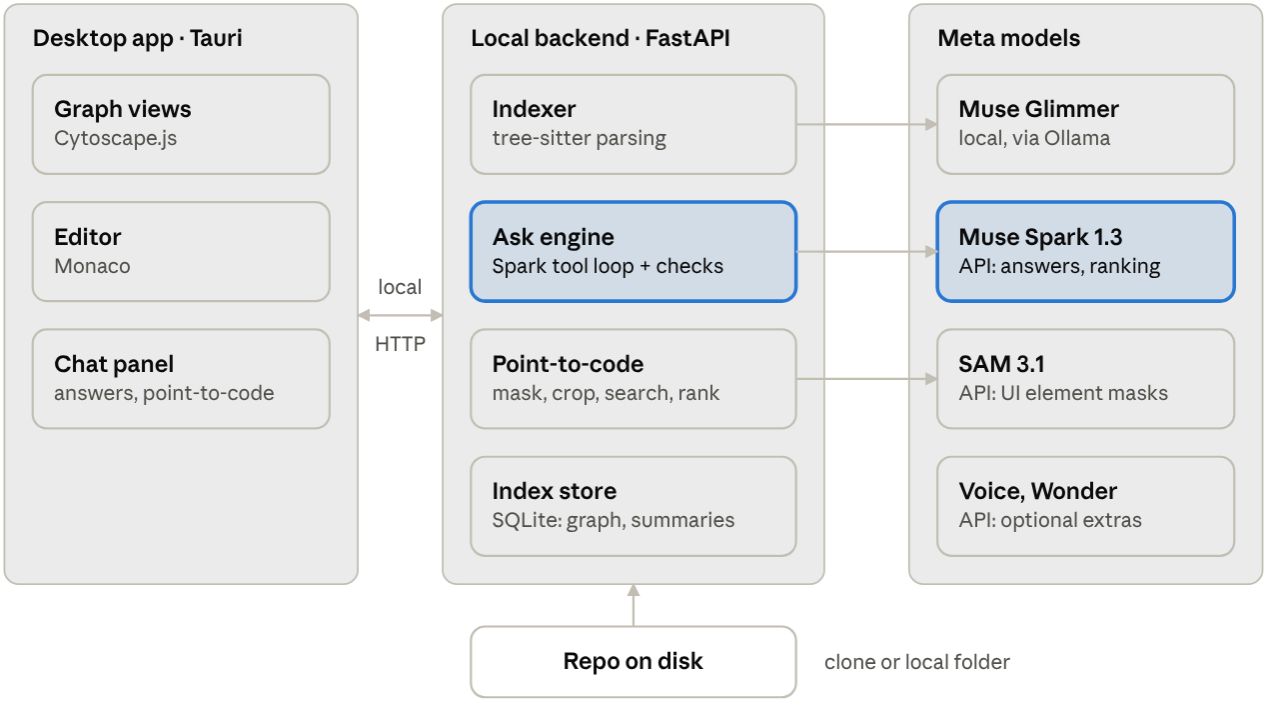
Job	Primary (Meta Model API)	Fallback (open weights)
Answers, tracing, summaries	Muse Spark 1.3	Muse Spark Open on Baseten, Fireworks or Groq
Private mode	Muse Glimmer on Modal	Same

Job	Primary (Meta Model API)	Fallback (open weights)
Point-to-code	SAM 3.1 API	SAM 3.1 open weights self-hosted on Modal

Architecture, tech stack and pipelines

A Tauri desktop app talks over local HTTP to a Python backend that parses the repo, stores the graph in SQLite and is the only part that calls Meta models.

The app talks to a local backend; only the backend calls Meta models



system architecture · 3 layers

Keeping all model calls in the backend keeps the API key out of the UI and makes Private mode a single switch: the backend sends work to Glimmer instead of Spark.

Tech stack

Layer	Choice	Why
Desktop shell	Tauri	Small, fast desktop app; can start the backend as a sidecar process
UI	React + Vite + TypeScript	Fast to build; good libraries for the panes

Layer	Choice	Why
Graph	Cytoscape.js with the fcose and ELK layouts	Handles large graphs, nested (compound) nodes for folders, built-in filtering and animation
Editor	Monaco (`@monaco-editor/react`)	The VS Code editor as a component
Backend	Python + FastAPI	Quick to write; good tree-sitter and AI client libraries
Parsing	tree-sitter (TS/JS, Python grammars)	Real syntax trees, fast, tolerant of broken code
Storage	SQLite	One local file; easy to cache by file hash
Meta API client	One OpenAI-compatible client; provider chosen in .env (Meta Model API, or an open-weights host)	Meta says existing OpenAI-compatible clients work
Local model	Ollama running Muse Glimmer	One command to run; OpenAI-compatible endpoint

Fallback: if Tauri packaging fights you, ship it as a local web app (same React UI in the browser, same backend). Nothing else changes.

Indexing pipeline (runs when a repo is opened)

1. **Fetch:** shallow-clone the GitHub URL, or read the local folder.
2. **Walk:** list files, skipping ignored ones; record folder, size, language and lines of code.
3. **Parse:** run tree-sitter on each file to extract functions, classes, imports, exports, call sites, backend route definitions (Express, FastAPI) and frontend `fetch` / `axios` calls.
4. **Link:** resolve each import to a file (including tsconfig path aliases and Python relative imports) to make file-to-file edges; match frontend calls to backend routes to make dashed route edges.
5. **Score:** compute how central each file is (PageRank) and flag entry points, tests and config.
6. **Summarise:** write a one-line summary per file, then per folder from its files' summaries (bottom-up). Spark in Cloud mode, Glimmer in Private mode. Cached by file hash so unchanged files are never re-summarised.
7. **Ready:** the UI loads the graph. On every save, only that file is re-parsed and re-linked.

Ask pipeline (runs for each question)

1. Send Spark the question, the folder summaries and a short repo overview.

2. Spark calls read-only tools: `search_code(text)`, `find_symbol(name)`, `read_lines(file, start, end)`, `neighbours(file)`. Up to about 8 tool calls per question.
3. Spark returns JSON: `answer` (short text) and `steps` (ordered list of file, start line, end line, claim, graph node).
4. The backend checks each step: file exists, lines exist, and the named function appears there. Failed steps are marked unverified.
5. The UI animates the path across the steps' nodes and shows citation chips.

Security and privacy

RepoBuddy treats every repository as untrusted input and never runs its code.

- **No code execution.** It parses files as text; it never installs dependencies, runs scripts or executes tests from a repo.
- **Prompt injection.** Repo text (comments, READMEs) can contain instructions aimed at the AI. File content is passed to Spark clearly marked as data, and the AI's only tools are read-only (search, read lines, list neighbours).
- **Keys stay local.** The Meta API key lives in the backend's environment or the OS keychain, never in the UI code or the repo.
- **No silent writes.** File changes happen only when the user accepts a diff. No git commits or pushes.
- **Limits.** Maximum file size (1 MB), file count (about 5,000) and context sent per question, to keep cost and latency bounded.
- **Data retention.** Cloned repos and the index stay in a local cache folder; a "Forget this repo" button deletes both.
- **What leaves the machine** is shown per mode (Cloud, Private, Hybrid), and Hybrid asks before sending any snippet.

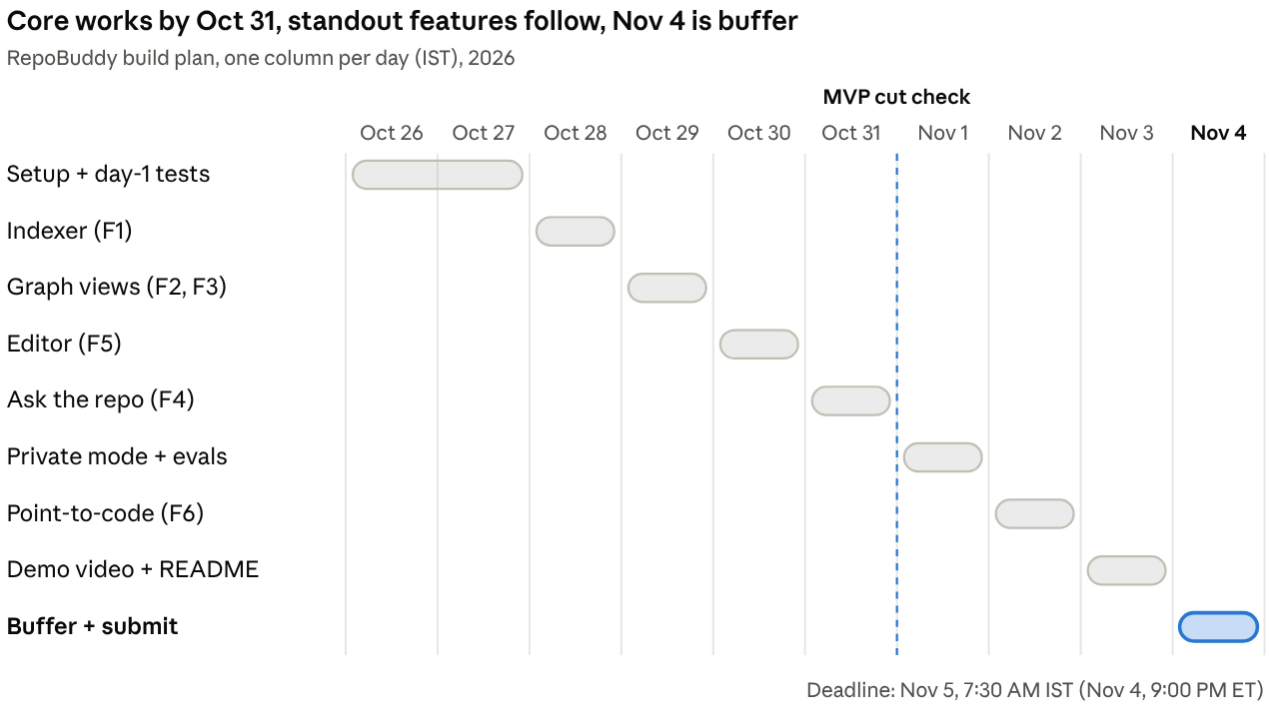
Scope

The MVP is F1 to F5 working end to end on real repos; everything else is added only once that path is solid.

Tier	What's in it	Rule
MVP (must ship)	Open repo (F1), Folder view (F2), Connections view with focus and filters (F3), Ask the repo with path highlighting and checks (F4), Monaco editor with save and graph links (F5), TS/JS + Python	If any of these is not working by Nov 1, cut everything below
Standout	Point-to-code (F6), Private mode with Glimmer (F7), eval numbers for the README	Build only after the MVP demo path works
Bonus	Voice questions (F8), Deep investigate (Muse Wonder), AI edit diffs, "Group by Features"	Only if time remains on Nov 2
Out	GitHub sign-in for private repos, other languages, terminal, debugger, git UI, extensions, running code, collaboration	Not in the hackathon build

Build plan (9 days)

The core (F1 to F5) is done by the end of Oct 31; Nov 1 is the cut check, and Nov 4 is kept empty as buffer.



build plan · Oct 26 to Nov 4

If the MVP path is not working at the cut check, Nov 1 and Nov 2 go to fixing it and point-to-code is dropped.

What "done" means each day

1. **Oct 26–27, setup and day-1 tests:** public repo created; API key working; one successful call each to Spark and SAM 3.1; Glimmer running locally (or the fallback decided); tree-sitter parses 3 real repos and the import edges look right on a spot check; SAM tested on 3 UI screenshots in the playground.
2. **Oct 28, indexer:** open a URL or folder → SQLite holds folders, files, symbols, import edges and route edges; re-index on save works.
3. **Oct 29, graph views:** Folder view drill-in and breadcrumb back-out; Connections view with focus mode, filters and search; both views linked.
4. **Oct 30, editor:** Monaco with file tree and tabs; open, edit, save; "Show in graph" and double-click-to-open both ways.
5. **Oct 31, ask the repo:** Spark tool loop, JSON steps, line checks, path animation and citation chips working on 5 questions.
6. **Nov 1, private mode and evals:** summaries filled in; Cloud/Private/Hybrid switch; write the 20-question eval and record the numbers.
7. **Nov 2, point-to-code:** screenshot → click → SAM mask → Spark crop reading → top 3 files; polish the demo path.
8. **Nov 3, video and README:** script, record, edit; README with setup, architecture, "Built during the hackathon" and eval numbers.

9. **Nov 4, buffer and submit:** fix only what breaks the demo; submit in the evening IST, hours before the deadline.

Demo video plan

A 2.5 to 3 minute video that shows the strongest moment first, uses a real well-known open-source repo, and labels which Meta model does each step.

1. **0:00–0:15 Hook.** A question is asked and a path lights up across the graph, step by step, ending in the editor on the exact lines. One-line voiceover: "Understand any codebase in minutes."
2. **0:15–0:35 Problem.** A quick scroll through a confusing repo and a hairball graph from a typical tool. "New code is slow to understand, and graphs like this don't help."
3. **0:35–1:10 Explore.** Paste the GitHub URL, the map builds, drill through the Folder view with summaries, switch to Connections, use focus mode.
4. **1:10–1:45 Ask.** "What happens when a user registers?" Show the path, click a step, show the verified citation chips. On-screen label: "Muse Spark 1.3, tool calling".
5. **1:45–2:15 Point-to-code.** Click a button in a screenshot; the outline appears, then the component opens. Label: "SAM 3.1 + Muse Spark".
6. **2:15–2:35 Private mode.** Flip the switch, show network off or the local model running, ask again. Label: "Muse Glimmer, on our own private GPU endpoint".
7. **2:35–2:55 Under the hood.** One architecture slide plus the eval numbers from the metrics table.
8. **2:55–3:00 Close.** Name, repo link, and "Built during the Meta AI Developer Hackathon, Oct 26 – Nov 4".

Record the screen at 1080p or higher, with the app at a large font size. Re-record any segment that lags; never fake a result.

Risks and compliance

The biggest risk is a graph that looks wrong on a real repo, so it is tested on day 1, before any UI.

Risk	Effect	Mitigation
Import resolution misses edges (path aliases, re-exports, dynamic imports)	Graph and paths look wrong	Test 3 real repos on day 1; support tsconfig `paths`; mark inferred edges as dashed

Risk	Effect	Mitigation
Spark cites wrong lines	Trust is lost	The check step; show "unverified" honestly; measure citation accuracy
SAM prompts don't fit UI elements well	Point-to-code fails	Playground test on day 1; fall back to a box around the click; it is an add-on, not core
Glimmer too slow or too big for your machine	Private mode fails in the demo	Quantized build, or self-host on Modal/Baseten credits and say so
Large graphs render slowly	Laggy demo	Folder view by default; "Core only" toggle; cap nodes shown at once
Judges ask "how is this different from Muse Code or Copilot?"	Lower impact score	Position as understanding and navigation, not code generation; lead with paths and point-to-code
Scope creep into IDE features	No time for the standout features	The Scope table's cut rule on Nov 1

Hackathon compliance checklist

- [ ] Public GitHub repo created on Oct 26, with regular commits (the history proves new work)
- [ ] README section "Built during the hackathon" listing everything new, plus third-party libraries used (Monaco, tree-sitter, Cytoscape.js, etc.)
- [ ] Only code, data and demo repos you have the rights to use (pick an open-source demo repo with a permissive license)
- [ ] Meta models do real, visible work (Stage 1 pass/fail)
- [ ] Demo shows only features built during the hackathon
- [ ] Video link opens in an incognito window
- [ ] Everyone who worked on it listed on the submission page
- [ ] Submitted by Nov 4 night IST (deadline Nov 5, 7:30 AM IST)

Sources

- Meta Global AI Developer Hackathon – Cerebral Valley
- Muse Spark 1.3

- Muse Glimmer
- SAM 3.1
- Muse Wonder
- Muse Spark Open
- Muse Voice Transcribe